UNSUPERVISED CUSTOMER SEGMENTATION IN A SMALL-MEDIUM BUSINESS: A CLUSTERING ANALYSIS

data-science.online

Prepared by:
Floriand BESHKU

Connecticut
USA

2026

PAGE INDEX

Project Summary 3

Business Context & Problem Definition 4

Data Description & Preparation 5

Methodology 7

Analysis & Results 9

Business Implications & Recommendation 14

Appendix (Technical & Codes) 15

PROJECT SUMMARY

This project demonstrates how data can be used to develop a clear and consistent understanding of customer behavior by replacing assumptions with an objective, data-driven segmentation approach. The goal is not only to group customers into clusters, but also to explain why customers fall into different segments and which behavioral factors truly separate one group from another.

For example, most coffee shop owners already recognize that some customers come in every morning for a quick drink, while others stay longer to work or socialize. What is often missing, however, is a precise and structured way to define these customer types using measurable evidence. This project bridges that gap by creating customer segments based on behaviors such as visits per month, average spend per visit, food versus drink ordering patterns, time spent in the shop, dine-in versus mobile ordering, weekend activity, promotion usage etc. Instead of relying on broad statements like “morning customers buy coffee quickly,” the analysis can identify clusters that quantify how much more frequent certain customers are, how much they spend relative to others, and how their ordering and in-store behavior differs.

The analysis is based on a dataset of 500 coffee shop customers, with each row representing one customer and multiple variables summarizing their typical purchasing and visit behavior. Before clustering, the data was reviewed and prepared to ensure consistency, and key variables were standardized so that all features contributed fairly to the methods used. Two clustering techniques were then applied: Hierarchical Clustering and K-Means. The results were evaluated using cluster profiles, centroid comparisons, and visual tools which help confirm if clusters are meaningful and well separated. If any of this sounds unfamiliar at first, there is no need to worry, this report is intentionally written in a simple, practical way so it remains understandable for both technical and non-technical readers.

At the end this project produces both interpretable customer segments and actionable insights. The final clusters can support targeted marketing, promotion design, staffing decisions, and menu strategy. Although the analysis is presented in a coffee shop setting, the same approach is widely applicable: any small or medium business that tracks customer activity (such as retail, restaurants, gyms, salons, or service providers) can use this framework to understand customer types, personalize offers, improve operations, and support sustainable growth.

BUSINESS CONTEXT & PROBLEM DEFINITION

In many small and medium businesses, customer understanding is still based on informal observation, staff experience, or general intuition. While this may work at, it often leads to broad assumptions about customer types, inconsistent marketing efforts, and limited ability to explain why certain strategies work for some customers but not for others. As the business grows, adds new promotions, expands operating hours, or increases digital ordering, these limitations become more important because decisions need to be consistent, measurable, and scalable.

This type of analysis is particularly useful for small/medium businesses that rely on repeat customers and daily operating efficiency. A coffee shop (considered in this project) can use clustering to identify distinct customer groups and tailor decisions such as menu bundles, promotion design, staffing coverage, store layout, and mobile ordering workflows. For example, a shop may want to differentiate between customers who visit frequently for quick purchases, customers who buy food and spend more per visit, and customers who stay longer to work or study. Marketing teams can use segmentation to run more targeted campaigns instead of applying the same discount to everyone. Operations managers can use customer clusters to anticipate peak demand patterns and align staffing levels, prep schedules, and inventory with real customer behavior. Over time, clustering can also support customer relationship efforts by helping the business understand which groups most valuable, which groups are are most likely to respond to promotions, and which groups have the highest potential for retention or upselling.

The main question addressed by this project is: Can customers be grouped into meaningful segments based on observable behavior, and which behaviors (frequency, spend, dwell time, food preference, dine-in vs. mobile ordering, promotion usage, etc) most strongly differentiate those segments to support better business decisions?

DATA DESCRIPTION & PREPARATION

The analysis is based on a behavioral dataset of 500 coffee shop customers, where each record represents one customer and summarizes their typical purchasing and visit patterns. The dataset is designed to reflect realistic coffee shop behavior, including differences between quick morning visits, dine-in customers, weekend visitors, and longer "work session" customers. The dataset includes three main types of information:

1) Customer identifiers and reference labels

- customer_id: unique customer ID (used only for tracking, not clustering)
- segment_true: a reference "ground truth" segment used for realism and sanity-checking (not used as an input to clustering)

2) Preference variables (categorical)

- daypart_mode, daypart_code: most common visit time (Morning, Afternoon, Evening)
- favorite_drink, favorite_drink_code
- favorite_food, favorite_food_code

These variables describe what customers tend to order, but they were not used in the main clustering input to keep the distance-based models focused on numeric behavior.

3) Behavioral variables (numeric) used for clustering

The clustering was performed on measurable behaviors that directly describe customer value and operational impact:

- visits_per_month
- avg_ticket_usd
- avg_drinks_per_visit
- avg_food_items_per_visit
- pct_food_orders
- avg_dwell_minutes
- pct_mobile_orders
- pct_dine_in
- discount_redemption_rate
- weekend_share_pct
- avg_party_size
- work_session_pct

These features capture **frequency**, **spending**, **basket composition**, **time-in-store**, **channel preference**, and **promotion sensitivity**, which are core drivers of customer segmentation in a coffee shop setting.

Header of the dataset:

customer_id	segment_true	daypart_mode	daypart_code	favorite_drink	favorite_drink_code	favorite_food
C0001	Morning Commuters (Grab & Go)	Morning	1	Espresso	6	None
C0002	Morning Commuters (Grab & Go)	Morning	1	Drip Coffee	1	None
C0003	Morning Commuters (Grab & Go)	Morning	1	Americano	2	Pastry
C0004	Morning Commuters (Grab & Go)	Morning	1	Drip Coffee	1	Pastry
C0005	Morning Commuters (Grab & Go)	Morning	1	Latte	3	None

favorite_food_code	visits_per_month	avg_ticket_usd	avg_drinks_per_visit	avg_food_items_per_visit	pct_food_orders
0	20	6.61	1.06	0.14	4.9
0	13	5.21	0.92	0.22	27
1	22	4.07	0.96	0.16	5.2
1	23	6.47	0.83	0	0
0	8	5.39	0.92	0.33	23.4

avg_dwell_minutes	pct_mobile_orders	pct_dine_in	discount_redemption_rate	weekend_share_pct	avg_party_size
7	75	25.2	22.6	22.5	1.3
2	50.6	10.3	2.7	18	1.1
9	36.4	24	20.3	10.5	1.3
7	54	9.3	25.4	15.6	1
3	73.9	11	21.7	16.2	1.1

METHODOLOGY

The goal is to **group coffee shop customers into clear segments** based on how they actually behave, how often they visit, what they typically buy, whether they stay or leave quickly, and how they place orders (mobile vs. in-store). These segments help the business make better decisions in areas like promotions, staffing, menu design, and customer retention.

1) What information is used to group customers.

Instead of using personal information, we focused only on observable shopping behavior, such as:

- How often do customers visit (visits per month)
- How much they typically spend per visit (average ticket)
- What they usually buy (drinks vs. food, how many items per visit)
- How long they stay (quick visits vs. longer stays)
- How they order (mobile ordering vs. dine-in)
- How price-sensitive they are (how often they redeem discounts)
- When they visit (weekday vs. weekend patterns)
- Group behavior (average party size and likelihood of “work session” visits)

This set of variables captures the main behaviors that matter for a coffee shop: revenue value, speed/traffic flow, product mix, and customer experience style.

2) Making the comparisons fair (Standardization)

Because these measures use different units (dollars, minutes, percentages, counts), we first applied a step called standardization. Standardization puts every variable on a comparable scale so that:

- spending doesn’t overpower minutes just because dollars are numerically larger,
- and no single feature “wins” simply because of how it is measured.

This helps ensure the clustering is based on true behavioral similarity, not measurement scale.

3) Approaches used to find customer groups.

To strengthen confidence in the segmentation, two clustering methods are applied and compared their outputs:

A) **Hierarchical clustering (tree-based grouping)**

This method starts with each customer as their own group and then repeatedly merges the most similar customers. The result is displayed as a tree diagram (dendrogram), which helps us see how naturally the groups form and supports the choice of how many segments to use.

B) **K-means clustering (grouping around typical customer types)**

This method creates a "typical customer profile" (a cluster center) for each segment and assigns each customer to the closest profile. It repeats until the groups stabilize. This approach is widely used in customer segmentation because it produces clear, stable clusters that are easy to explain and act on.

4) Choosing the number of customer segments

We reviewed the clustering outputs and tested different numbers of groups. The final choice aimed to balance:

- clarity (segments should be easy to interpret), and
- actionability (segments should be useful for real business decisions).

Based on the patterns in the results, we focused on a 3-cluster solution that produced distinct customer types.

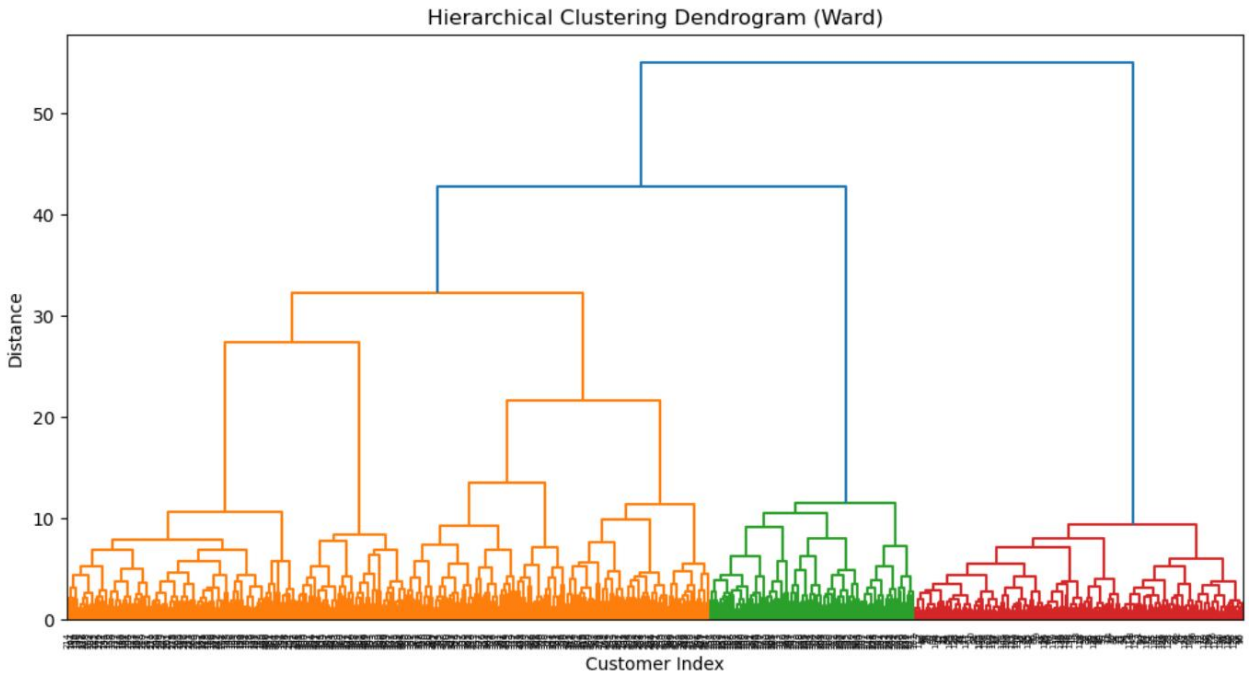
5) Turning clusters into business profiles

After clustering, we summarized each segment by comparing average behaviors (for example: which cluster visits most, which stays longer, which is most mobile-first, which is most price-sensitive). This is what converts the model output into real customer profiles that the business can use.

ANALYSIS & RESULTS

A. Hierarchical clustering

Hierarchical clustering is a method that groups customers by building clusters step by step based on similarity.



The hierarchical clustering dendrogram shows how customers are gradually grouped together based on similarity in their behaviors. Each customer starts as an individual point, and the algorithm repeatedly merges the most similar customers or customer groups until larger segments are formed. The vertical height of each merge represents the "distance," meaning how different two groups were when they were combined, lower merges indicate very similar customers, while higher merges indicate more distinct groups.

In this dendrogram, the structure separates naturally into three main branches, and the final merges occur at relatively high distances, indicating that the groups being combined near the top are substantially different. Based on this clear separation, the dendrogram is cut at k = 3 to produce three customer segments for further analysis and interpretation.

```
hier_cluster
1    140
2     87
3    273
Name: count, dtype: int64
```

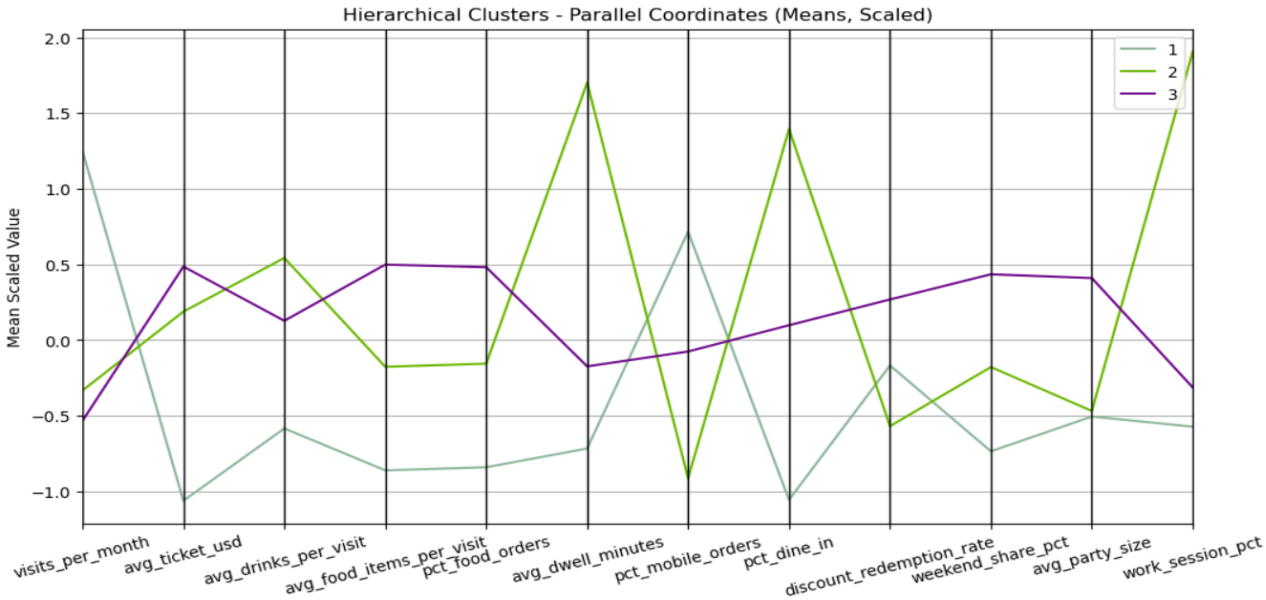
Using the hierarchical clustering results, the dendrogram is "cut" into three customer groups: Cluster 1 (140 customers), Cluster 2 (87 customers), and Cluster 3 (273 customers).

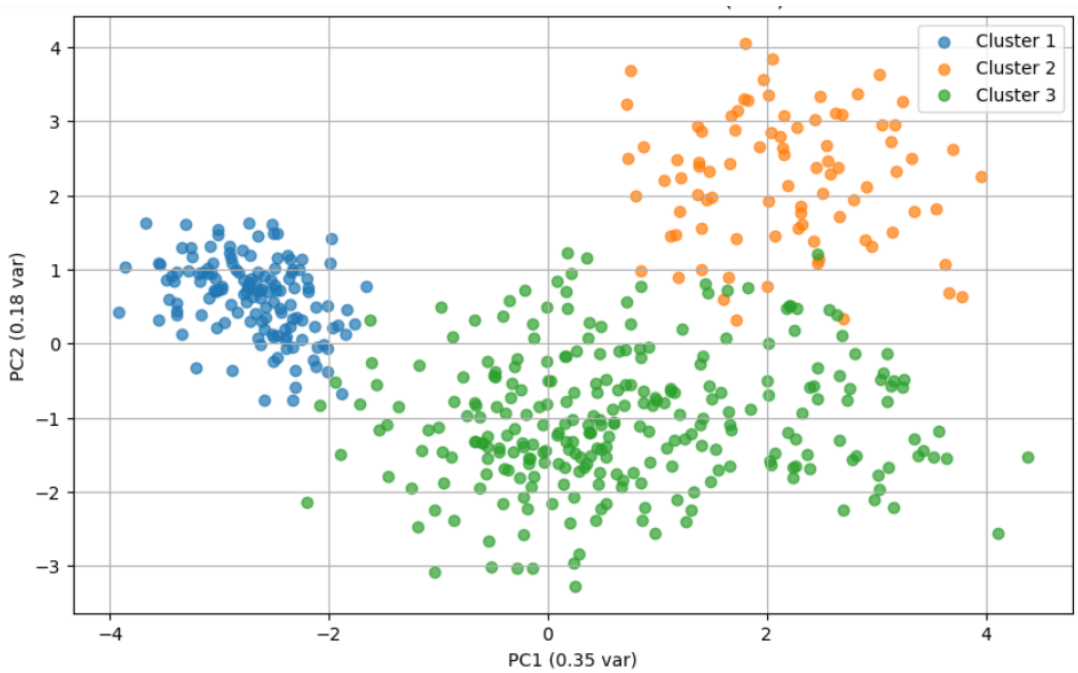
hier_cluster	visits_per_month	avg_ticket_usd	avg_drinks_per_visit	avg_food_items_per_visit	pct_food_orders	avg_dwell_minutes
1	1.250807	-1.062798	-0.58464	-0.861809	-0.841397	-0.717361
2	-0.333954	0.18743	0.542512	-0.177139	-0.156442	1.701359
3	-0.535015	0.485294	0.126927	0.498404	0.481341	-0.174314

pct_mobile_orders	pct_dine_in	discount_redemption_rate	weekend_share_pct	avg_party_size	work_session_pct
0.716385	-1.056439	-0.168336	-0.735858	-0.506095	-0.573225
-0.913638	1.394206	-0.569575	-0.17886	-0.469145	1.907747
-0.076218	0.097456	0.267839	0.434362	0.409044	-0.314002

The table shows average standardized (z-score) values for each behavior metric, where positive numbers mean "above the overall customer average" and negative numbers mean "below average."

Based on those standardized profiles, **Cluster 1** represents high-frequency, low-ticket, quick visits (more visits per month, lower average ticket, fewer food and drink items, shorter dwell time, higher mobile-order share, and lower dine-in), which fits a grab-and-go routine pattern. **Cluster 2** looks like sit-and-stay customers (much higher dwell time, strongly higher dine-in, lower mobile orders, and slightly higher ticket with more drinks), which aligns with people who spend time in the café (work/study/meetings) rather than stopping briefly. **Cluster 3** is the largest group and appears more food-led and higher basket (higher ticket, more food items, and higher share of food orders, with near-average channel mix and slightly shorter dwell), which fits customers who buy fuller orders but visit less frequently than Cluster 1.





This PCA chart is a 2-D "map" of customers created by summarizing all clustering variables into two combined axes: PC1 (about 35% of the overall variation) and PC2 (about 18%). Customers plotted close together show similar overall behavior, while customers farther apart behave more differently. In this view,

Cluster 1 (blue) forms a tight group on the left, indicating a consistent segment that is mainly separated from others along PC1.

Cluster 2 (orange) appears in the upper-right and is also well separated, suggesting a distinct behavior pattern and the most clearly differentiated segment in the plot.

Cluster 3 (green) is the largest and most spread out across the middle to lower area, which typically reflects a more mixed segment with greater variation inside it; it also shows some overlap near the boundary with Cluster 1 and 2, meaning a small subset of customers shares similar behaviors across those groups. The PCA visualization supports the presence of three meaningful segments, with Cluster 1 and 2 most distinct and Cluster 3 capturing the broadest range of customer behavior.

Vocabulary:
PCA (Principal Component Analysis) is a method that simplifies a dataset with many variables into a few summary measures so it's easier to visualize and understand.

PCA chart is a simple "map" of customers after clustering. Each dot is one customer, and dots that are close together have similar overall behavior across the variables used (visits, ticket size, dwell time, mobile vs. dine-in, food share, party size, work sessions, etc.). The two axes (PC1 and PC2) are summary dimensions that compress all those behaviors into a 2D view.

B. K-Means Clustering

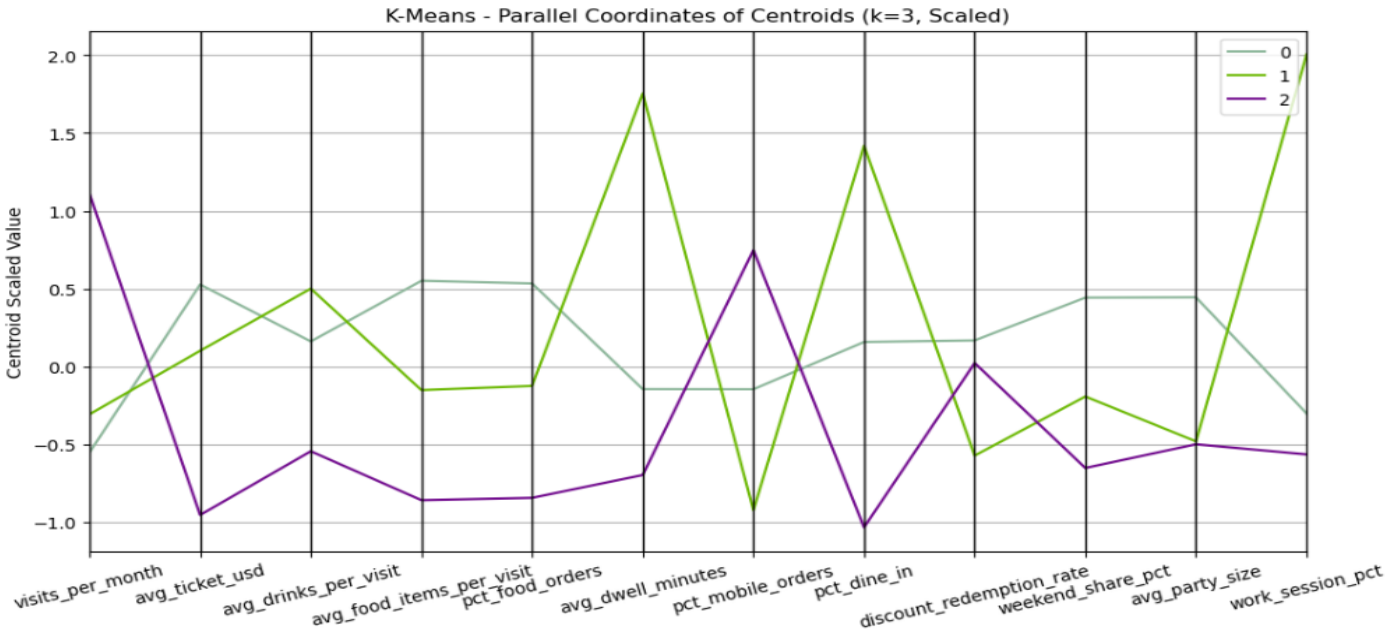
```
kmeans_cluster
0    263
1     83
2    154
Name: count, dtype: int64
```

K-Means is another segmentation method, but instead of building a "family tree" like hierarchical clustering (in A), it directly creates k groups by placing k centers (called centroids) and assigning each customer to the nearest center based on overall behavior similarity. The algorithm then updates the centers repeatedly until the group definitions stabilize.

K-Means clustering technique in our case produced these groups: Cluster 0 with 263 customers, Cluster 1 with 83 customers, and Cluster 2 with 154 customers (very similar to the first technique used in A). The centroid table summarizes the "typical customer" in each segment using standardized values (0 = average, positive = above average, negative = below average).

Cluster	visits_per_month	avg_ticket_usd	avg_drinks_per_visit	avg_food_items_per_visit	pct_food_orders	avg_dwell_minutes
0	-0.553025	0.526571	0.160989	0.550783	0.533467	-0.145328
1	-0.307174	0.101197	0.500247	-0.151742	-0.124741	1.753895
2	1.110007	-0.953816	-0.544549	-0.858839	-0.843821	-0.697091

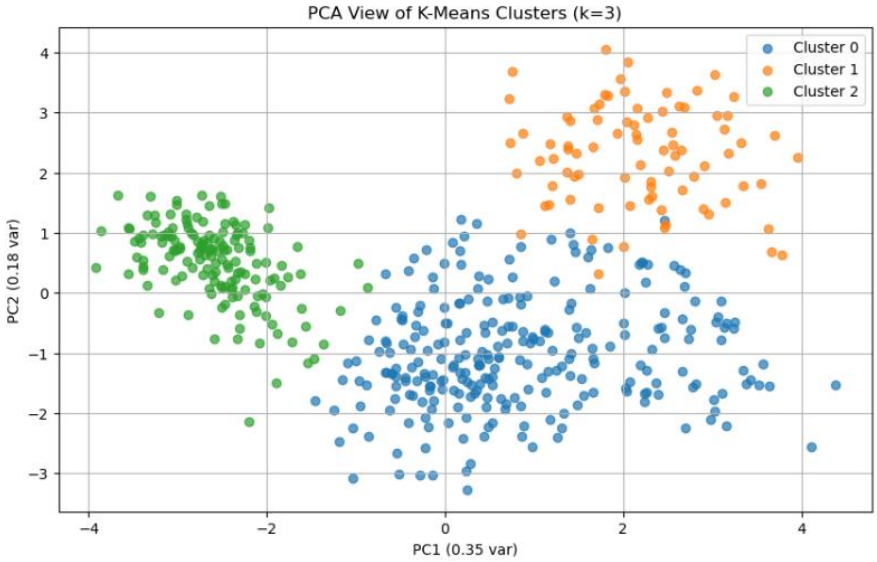
pct_mobile_orders	pct_dine_in	discount_redemption_rate	weekend_share_pct	avg_party_size	work_session_pct
-0.146255	0.157538	0.167558	0.44259	0.444378	-0.302089
-0.923257	1.418515	-0.572396	-0.192728	-0.481001	2.005301
0.747372	-1.033566	0.022345	-0.65198	-0.499664	-0.564873



Cluster 0 is the food-forward, higher-ticket occasional segment, they visit less often than average but have higher ticket values and higher food-related measures, suggesting more purposeful, higher-value customers.

Cluster 1 is the dine-in, long-stay segment, very high dwell time, high dine-in share, low mobile ordering, and drink-focused behavior, matching customers who treat the coffee shop as a place to sit.

Cluster 2 is the frequent, mobile, quick-visit segment, higher visits per month, lower spending per visit, lower food ordering, shorter dwell time, higher mobile ordering, and low dine-in (classic commuters, grab-and-go behavior).



The clusters are visibly separated, which supports that the segmentation is capturing real behavioral differences: Cluster 2 (green) forms a tight group on the left, suggesting a consistent type of customer behavior; Cluster 1 (orange) is clearly grouped in the upper-right, indicating a distinct segment (often consistent with longer-stay / dine-in patterns in your centroid results); and Cluster 0 (blue) is broader and more spread out in the middle/lower region, which usually means it represents a larger "general population" segment with more variety inside it. The small amount of overlap between colors is normal, some customers naturally sit near the "border" between two segments, but in general, the plot shows strong separation, which is a good sign for using these clusters in a business-facing segmentation story.

BUSINESS IMPLICATIONS & RECOMMENDATIONS

Based on the clustering process, the customer base separates into three clear behavioral groups that translate directly into business actions. Is considered the B approach for business insight and recommendations.

The largest group, Cluster 0 (with 263 customers), visits less frequently but spends more per visit, orders more food, shows stronger weekend activity, and tends to arrive in slightly larger parties. This looks like a "weekend/social treat" segment, and it offers a strong opportunity to grow revenue by enhancing the experience and increasing add-ons. Recommended actions include promoting weekend-focused bundles, strengthening food attachment with clear pairing suggestions (dessert + specialty drink, sandwich + iced drink), and rotating seasonal or limited-time items to create urgency and "treat" appeal. Since these customers often come with others, group-friendly tactics work well, such as shareable items, "bring-a-friend" offers, and small incentives for multi-item baskets (e.g., discount or free upgrade when purchasing 3+ items). To protect margins, incentives should focus on value bundles and upgrades (size upgrades, extra shots, etc.) rather than broad discounts. Operationally, this segment tends to concentrate demand on weekends, so improving weekend readiness fast food prep, visible pastry displays, quick table turnover, and adequate staffing helps capture demand without long waits that reduce repeat visits.

Cluster 1 (83 customers) is characterized by very long dwell time, strong dine-in behavior, high "work session" tendency, and low mobile ordering, indicating work/study regulars who value seating, Wi-Fi, and comfort. The strategy is to protect the work-friendly experience while increasing revenue per hour through long-stay offers such as a "study bundle" (drink + pastry/sandwich), timed add-on prompts (e.g., a discounted second drink after 60–90 minutes), and refill/upgrade options that feel convenient rather than pushy. Operationally, creating clear seating zones (work-friendly vs. quick-turn tables) and ensuring reliable Wi-Fi and outlets helps retain this segment without reducing capacity during peak periods.

Cluster 2 (154 customers) visits much more frequently but has a lower ticket, fewer food purchases, shorter dwell time, and higher mobile ordering, which fits a frequent grab-and-go routine segment. The best opportunities are to keep the experience fast and effortless while increasing basket size through one-click add-ons, smart upgrades (size or extra shot), and frequency-based loyalty or subscription offers that reward routine behavior without relying on heavy discounts. Operationally, this segment benefits most from a reliable mobile ordering flow, clear pickup organization, accurate ready times, and staffing aligned to peak rush periods.

The results suggest that promotions and operations should be tailored by segment: experience-driven weekend offers for Cluster 0, work-friendly bundles and long-stay monetization for Cluster 1, and speed with frictionless upselling for Cluster 2.

Appendix (Technical & Codes used for this analysis)

```
[1]: import os
     os.environ["OMP_NUM_THREADS"] = "1"

     import pandas as pd
     import numpy as np
     import matplotlib.pyplot as plt
     from pandas.plotting import parallel_coordinates

     from sklearn.preprocessing import StandardScaler
     from sklearn.cluster import KMeans
     from sklearn.decomposition import PCA

     from scipy.cluster.hierarchy import dendrogram, linkage, fcluster

     pd.set_option("display.max_columns", None)
```

```
[2]: df = pd.read_csv("coffee_shop_customers.csv")
     df.shape
```

```
[2]: (500, 20)
```

```
[3]: df.head()
```

[3]:

	customer_id	segment_true	daypart_mode	daypart_code	favorite_drink	favorite_drink_code	favorite_food	favorite_food_code	visits_per_month	avg_ticket_usd	avg
0	C0001	Morning Commuters (Grab & Go)	Morning	1	Espresso	6	NaN	0	20	6.61	
1	C0002	Morning Commuters (Grab & Go)	Morning	1	Drip Coffee	1	NaN	0	13	5.21	
2	C0003	Morning Commuters (Grab & Go)	Morning	1	Americano	2	Pastry	1	22	4.07	
3	C0004	Morning Commuters (Grab & Go)	Morning	1	Drip Coffee	1	Pastry	1	23	6.47	
4	C0005	Morning Commuters (Grab & Go)	Morning	1	Latte	3	NaN	0	8	5.39	

```
[3]: df.head()
```

[3]:

avg_drinks_per_visit	avg_food_items_per_visit	pct_food_orders	avg_dwell_minutes	pct_mobile_orders	pct_dine_in	discount_redemption_rate	weekend_share_pct	avg_part
1.06	0.14	4.9	7	75.0	25.2	22.6	22.5	
0.92	0.22	27.0	2	50.6	10.3	2.7	18.0	
0.96	0.16	5.2	9	36.4	24.0	20.3	10.5	
0.83	0.00	0.0	7	54.0	9.3	25.4	15.6	
0.92	0.33	23.4	3	73.9	11.0	21.7	16.2	

[4]:
```
cols_for_clustering = [
    "visits_per_month",
    "avg_ticket_usd",
    "avg_drinks_per_visit",
    "avg_food_items_per_visit",
    "pct_food_orders",
    "avg_dwell_minutes",
    "pct_mobile_orders",
    "pct_dine_in",
    "discount_redemption_rate",
    "weekend_share_pct",
    "avg_party_size",
    "work_session_pct"
]

X = df[cols_for_clustering].copy()
X.head()
```

[4]:

	visits_per_month	avg_ticket_usd	avg_drinks_per_visit	avg_food_items_per_visit	pct_food_orders	avg_dwell_minutes	pct_mobile_orders	pct_dine_in	discount_redempti
0	20	6.61	1.06	0.14	4.9	7	75.0	25.2	
1	13	5.21	0.92	0.22	27.0	2	50.6	10.3	
2	22	4.07	0.96	0.16	5.2	9	36.4	24.0	
3	23	6.47	0.83	0.00	0.0	7	54.0	9.3	
4	8	5.39	0.92	0.33	23.4	3	73.9	11.0	

[4]:

food_items_per_visit	pct_food_orders	avg_dwell_minutes	pct_mobile_orders	pct_dine_in	discount_redemption_rate	weekend_share_pct	avg_party_size	work_session_pct
0.14	4.9	7	75.0	25.2	22.6	22.5	1.3	12.1
0.22	27.0	2	50.6	10.3	2.7	18.0	1.1	3.1
0.16	5.2	9	36.4	24.0	20.3	10.5	1.3	3.8
0.00	0.0	7	54.0	9.3	25.4	15.6	1.0	2.1
0.33	23.4	3	73.9	11.0	21.7	16.2	1.1	1.5

[5]:
```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_scaled = pd.DataFrame(X_scaled, columns=cols_for_clustering)
X_scaled.head()
```

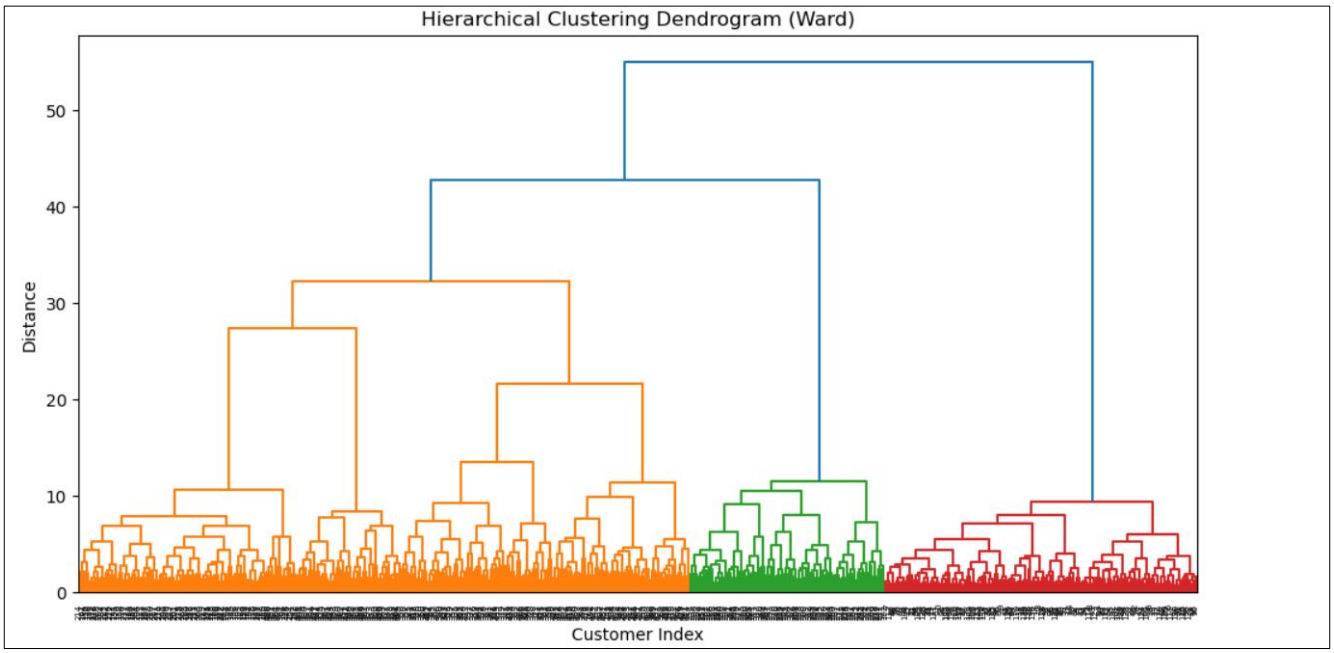
[5]:

	visits_per_month	avg_ticket_usd	avg_drinks_per_visit	avg_food_items_per_visit	pct_food_orders	avg_dwell_minutes	pct_mobile_orders	pct_dine_in	discount_redempti
0	1.611649	-0.893428	-0.536083	-0.914921	-1.252308	-0.700133	1.598078	-0.557007	0
1	0.463517	-1.437723	-0.970661	-0.710504	-0.316935	-0.817219	0.543578	-1.054578	-1
2	1.939686	-1.880934	-0.846496	-0.863817	-1.239611	-0.653298	-0.070107	-0.597079	-0
3	2.103705	-0.947858	-1.250032	-1.272652	-1.459698	-0.700133	0.690516	-1.087972	0
4	-0.356577	-1.367742	-0.970661	-0.429430	-0.469303	-0.793802	1.550539	-1.031202	-0

[5]:

food_items_per_visit	pct_food_orders	avg_dwell_minutes	pct_mobile_orders	pct_dine_in	discount_redemption_rate	weekend_share_pct	avg_party_size	work_session_pct
-0.914921	-1.252308	-0.700133	1.598078	-0.557007	0.033132	-0.463039	-0.168132	-0.316127
-0.710504	-0.316935	-0.817219	0.543578	-1.054578	-1.106770	-0.702454	-0.608269	-0.686560
-0.863817	-1.239611	-0.653298	-0.070107	-0.597079	-0.098616	-1.101478	-0.168132	-0.657749
-1.272652	-1.459698	-0.700133	0.690516	-1.087972	0.193520	-0.830142	-0.828338	-0.727719
-0.429430	-0.469303	-0.793802	1.550539	-1.031202	-0.018422	-0.798220	-0.608269	-0.752415

```
[6]: linked = linkage(X_scaled, method="ward")

     plt.figure(figsize=(12, 6))
     dendrogram(linked, distance_sort="descending", show_leaf_counts=True)
     plt.title("Hierarchical Clustering Dendrogram (Ward)")
     plt.xlabel("Customer Index")
     plt.ylabel("Distance")
     plt.show()
```




```
[7]: k = 3
     clusters_h = fcluster(linked, t=k, criterion="maxclust")

     df["hier_cluster"] = clusters_h
     X_scaled["hier_cluster"] = clusters_h

     df["hier_cluster"].value_counts().sort_index()
```

```
[7]: hier_cluster
     1    140
     2     87
     3    273
     Name: count, dtype: int64
```

```
[8]: X_scaled.groupby("hier_cluster")[cols_for_clustering].mean()
```

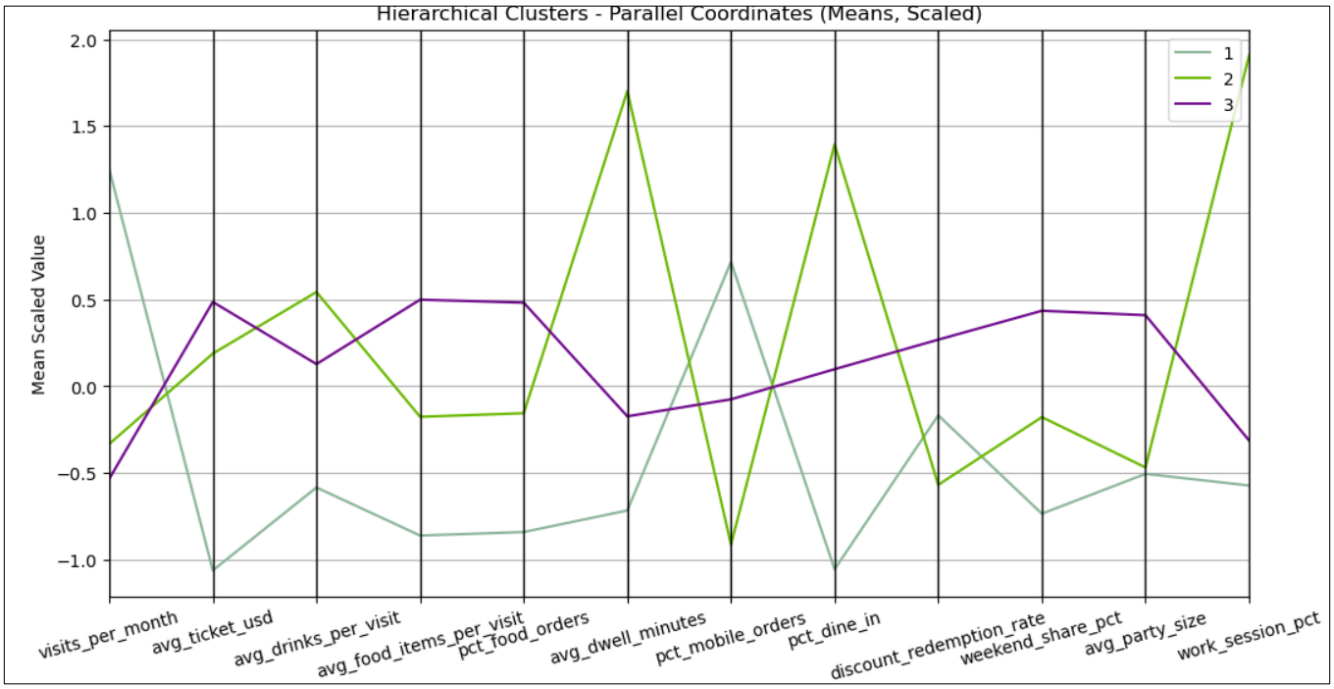
[8]:

hier_cluster	visits_per_month	avg_ticket_usd	avg_drinks_per_visit	avg_food_items_per_visit	pct_food_orders	avg_dwell_minutes	pct_mobile_orders	pct_dine_in	discount
1	1.250807	-1.062798	-0.584640	-0.861809	-0.841397	-0.717361	0.716385	-1.056439	
2	-0.333954	0.187430	0.542512	-0.177139	-0.156442	1.701359	-0.913638	1.394206	
3	-0.535015	0.485294	0.126927	0.498404	0.481341	-0.174314	-0.076218	0.097456	

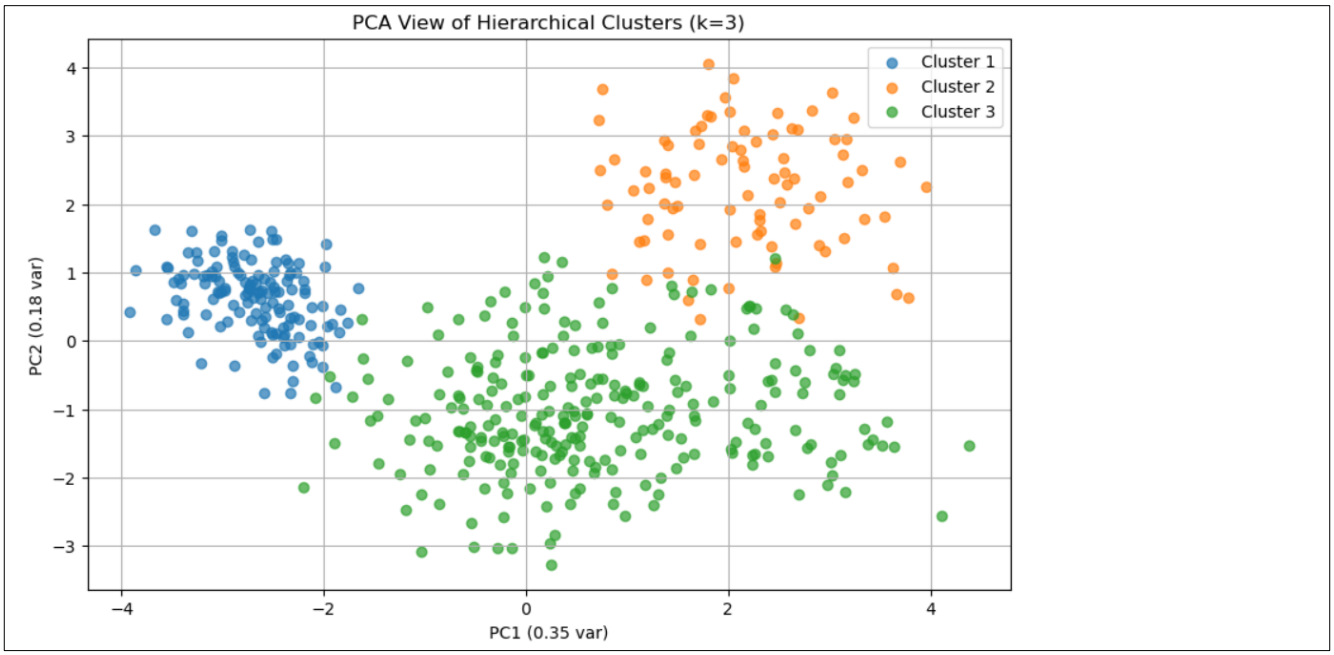
[8]:

_food_items_per_visit	pct_food_orders	avg_dwell_minutes	pct_mobile_orders	pct_dine_in	discount_redemption_rate	weekend_share_pct	avg_party_size	work_session_pct
-0.861809	-0.841397	-0.717361	0.716385	-1.056439	-0.168336	-0.735858	-0.506095	-0.573225
-0.177139	-0.156442	1.701359	-0.913638	1.394206	-0.569575	-0.178860	-0.469145	1.907747
0.498404	0.481341	-0.174314	-0.076218	0.097456	0.267839	0.434362	0.409044	-0.314002

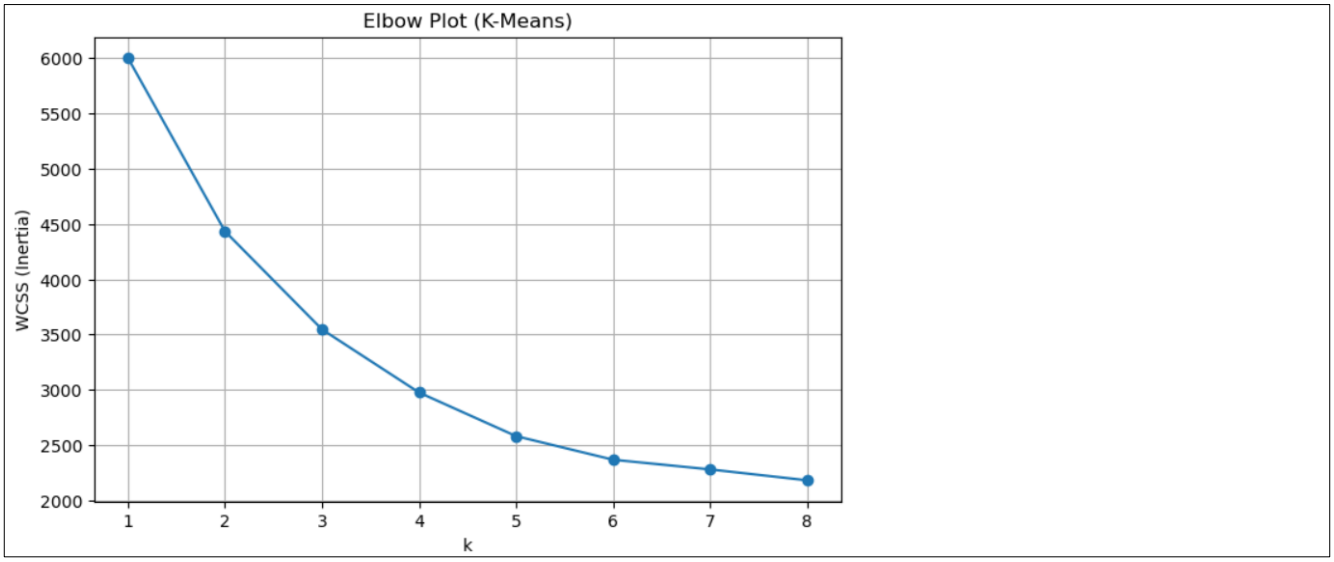
```
[9]: means_h = X_scaled.groupby("hier_cluster")[cols_for_clustering].mean().reset_index()

     plt.figure(figsize=(12, 6))
     parallel_coordinates(means_h, class_column="hier_cluster")
     plt.title("Hierarchical Clusters - Parallel Coordinates (Means, Scaled)")
     plt.ylabel("Mean Scaled Value")
     plt.xticks(rotation=15)
     plt.grid(True)
     plt.show()
```




```
[10]: pca = PCA(n_components=2)
      pc = pca.fit_transform(X_scaled[cols_for_clustering])

      pc_df = pd.DataFrame(pc, columns=["PC1", "PC2"])
      pc_df["hier_cluster"] = df["hier_cluster"].values

      plt.figure(figsize=(10, 6))
      for c in sorted(pc_df["hier_cluster"].unique()):
          tmp = pc_df[pc_df["hier_cluster"] == c]
          plt.scatter(tmp["PC1"], tmp["PC2"], label=f"Cluster {c}", alpha=0.7)

      plt.title(f"PCA View of Hierarchical Clusters (k={k})")
      plt.xlabel(f"PC1 ({pca.explained_variance_ratio_[0]:.2f} var)")
      plt.ylabel(f"PC2 ({pca.explained_variance_ratio_[1]:.2f} var)")
      plt.grid(True)
      plt.legend()
      plt.show()
```



[11]:
```python
wcss = []
K = range(1, 9)

for k in K:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X_scaled[cols_for_clustering])
    wcss.append(km.inertia_)

plt.figure(figsize=(8, 5))
plt.plot(list(K), wcss, marker="o")
plt.title("Elbow Plot (K-Means)")
plt.xlabel("k")
plt.ylabel("WCSS (Inertia)")
plt.grid(True)
plt.show()
```



[12]:
```
k_km = 3
kmeans = KMeans(n_clusters=k_km, random_state=42, n_init=10)

df["kmeans_cluster"] = kmeans.fit_predict(X_scaled[cols_for_clustering])

df["kmeans_cluster"].value_counts().sort_index()
```

[12]:
```
kmeans_cluster
0    263
1     83
2    154
Name: count, dtype: int64
```

[13]:
```
# Centroids on scaled variables
centroids_scaled = pd.DataFrame(kmeans.cluster_centers_, columns=cols_for_clustering)
centroids_scaled
```

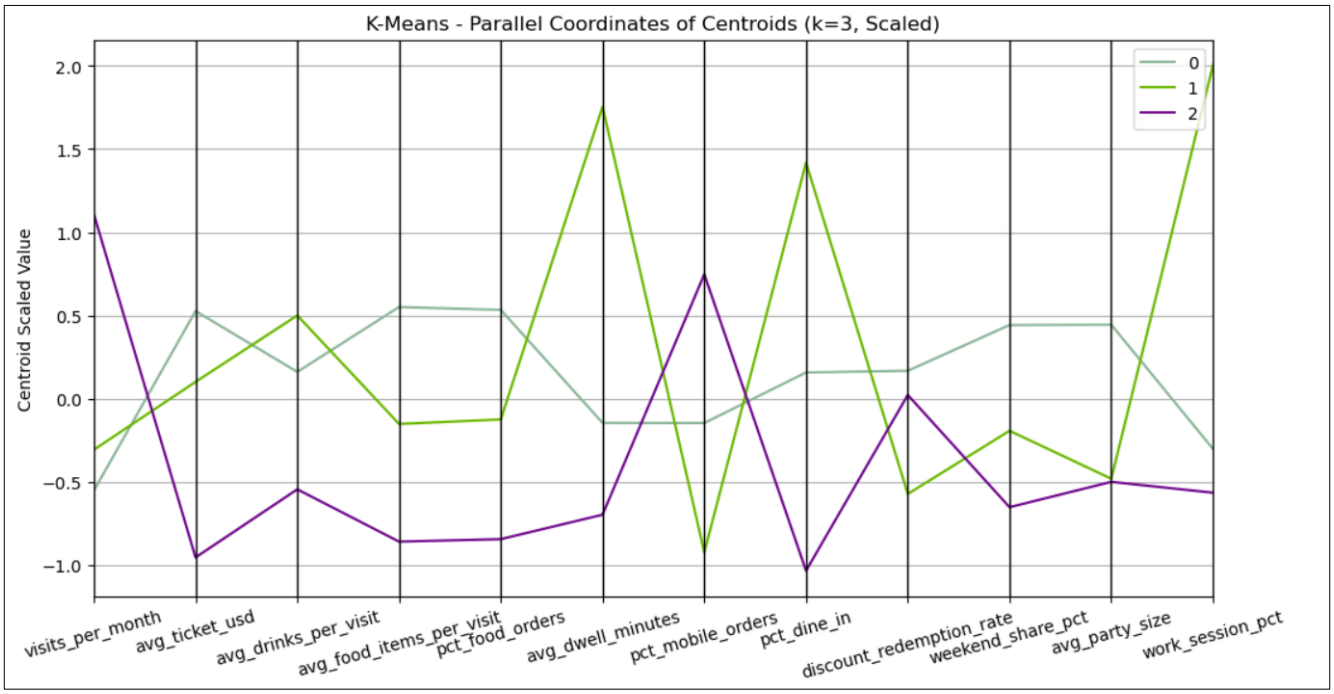
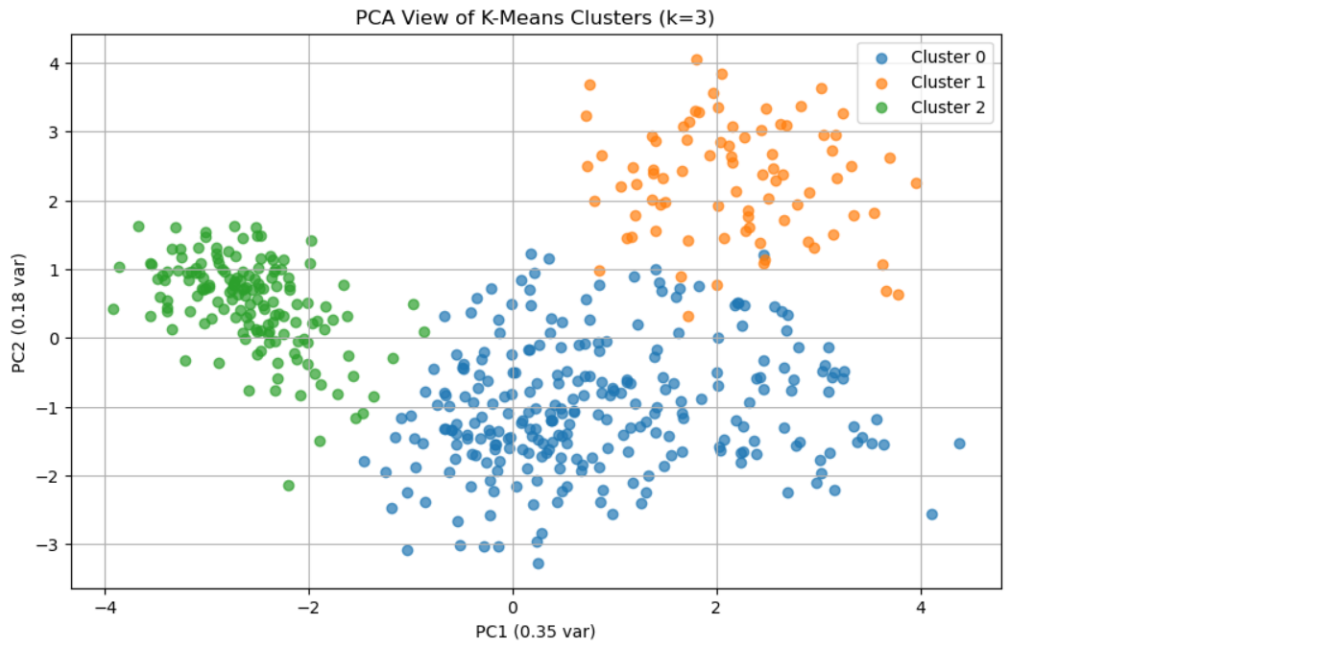
[13]:

	visits_per_month	avg_ticket_usd	avg_drinks_per_visit	avg_food_items_per_visit	pct_food_orders	avg_dwell_minutes	pct_mobile_orders	pct_dine_in	discount_redempti
0	-0.553025	0.526571	0.160989	0.550783	0.533467	-0.145328	-0.146255	0.157538	0
1	-0.307174	0.101197	0.500247	-0.151742	-0.124741	1.753895	-0.923257	1.418515	-0
2	1.110007	-0.953816	-0.544549	-0.858839	-0.843821	-0.697091	0.747372	-1.033566	0

[13]:

food_items_per_visit	pct_food_orders	avg_dwell_minutes	pct_mobile_orders	pct_dine_in	discount_redemption_rate	weekend_share_pct	avg_party_size	work_session_pct
0.550783	0.533467	-0.145328	-0.146255	0.157538	0.167558	0.442590	0.444378	-0.302089
-0.151742	-0.124741	1.753895	-0.923257	1.418515	-0.572396	-0.192728	-0.481001	2.005301
-0.858839	-0.843821	-0.697091	0.747372	-1.033566	0.022345	-0.651980	-0.499664	-0.564873

[14]:
```
centroids_plot = centroids_scaled.copy()
centroids_plot["kmeans_cluster"] = centroids_plot.index

plt.figure(figsize=(12, 6))
parallel_coordinates(centroids_plot, class_column="kmeans_cluster")
plt.title(f"K-Means - Parallel Coordinates of Centroids (k={k_km}, Scaled)")
plt.ylabel("Centroid Scaled Value")
plt.xticks(rotation=15)
plt.grid(True)
plt.show()
```



```
[15]: pca2 = PCA(n_components=2)
      pc2 = pca2.fit_transform(X_scaled[cols_for_clustering])

      pc2_df = pd.DataFrame(pc2, columns=["PC1", "PC2"])
      pc2_df["kmeans_cluster"] = df["kmeans_cluster"].values

      plt.figure(figsize=(10, 6))
      for c in sorted(pc2_df["kmeans_cluster"].unique()):
          tmp = pc2_df[pc2_df["kmeans_cluster"] == c]
          plt.scatter(tmp["PC1"], tmp["PC2"], label=f"Cluster {c}", alpha=0.7)

      plt.title(f"PCA View of K-Means Clusters (k={k_km})")
      plt.xlabel(f"PC1 ({pca2.explained_variance_ratio_[0]:.2f} var)")
      plt.ylabel(f"PC2 ({pca2.explained_variance_ratio_[1]:.2f} var)")
      plt.grid(True)
      plt.legend()
      plt.show()
```